26/05/2026



ASSIGNMENT 02
Business Intelligence & Database Applications

SQL

Autumn 2026

Benjamín Vallejo / / Section 01

Phase 1: Geographic Footprint

[Question 1]: I'm about to hop into a meeting about our national expansion. From how many different cities are the people in our database actually from? I need to know if we are truly "Brazil-wide" yet.

Code:

```
-- Q1 SQL05
SELECT COUNT(DISTINCT cus.customer_city) AS n_city
FROM Customers AS cus

- Manual check
--SELECT DISTINCT cus.customer_city AS city_name
--FROM Customers AS cus
```

Result:

```
1   USE Olist
2
3   -- Q1 SQL05
4   SELECT COUNT(DISTINCT cus.customer_city) AS n_city
5   FROM Customers AS cus
6
7   --SELECT DISTINCT cus.customer_city AS city_name
8   --FROM Customers AS cus
9
```

Results | Messages

	n_city
1	4119

Source:

- SQL01
- SQL05

[Question 2]: There's a massive logistics strike happening specifically in Minas Gerais (MG). To plan our alternative routes, I need a list of all cities in the database that are NOT in MG. Show me the name and state, and keep it in ascending order so I can scroll through it quickly.

Code:

```sql
--Q2
SELECT cus.customer_state,
       cus.customer_city
FROM Customers AS cus
WHERE cus.customer_state NOT IN ('MG')
ORDER BY cus.customer_city ASC
```

Result:

```sql
10  --Q2 where is order by in lectures
11  SELECT cus.customer_state,
12         cus.customer_city
13  FROM Customers AS cus
14  WHERE cus.customer_state NOT IN ('MG')
15  ORDER BY cus.customer_city ASC
16
```

Results Messages

	customer_state	customer_city
1	GO	abadiania
2	PA	abaetetuba
3	PA	abaetetuba
4	PA	abaetetuba
5	PA	abaetetuba
6	PA	abaetetuba
7	PA	abaetetuba
8	PA	abaetetuba
9	PA	abaetetuba
10	PA	abaetetuba
11	PA	abaetetuba
12	PA	abaetetuba
13	CE	abaiara
14	CE	abaiara
15	BA	abaira
16	BA	abaira
17	BA	abare
18	BA	abare
19	PR	abatia
20	PR	abatia

Source:

- SQL01

[Question 3]: We are planning a "São Paulo & Friends" marketing blitz. Give me a list of all the cities whose names start with "sao." I need the Name, State, Latitude, and Longitude so the design team can plot them on a heatmap.

Code:

```
-- Q3 SQL 1
SELECT geo.geolocation_city,
        geo.geolocation_state,
        geo.geolocation_lat,
        geo.geolocation_lng
FROM Geolocation AS geo
WHERE geo.geolocation_city like 'sao%'
```

Result:

```sql
-- Q3 SQL 1
SELECT geo.geolocation_city,
       geo.geolocation_state,
       geo.geolocation_lat,
       geo.geolocation_lng
FROM Geolocation AS geo
WHERE geo.geolocation_city like 'sao%'
```

Results Messages

	geolocation_city	geolocation_state	geolocation_lat	geolocation_lng
1	sao miguel de taipu	PB	-7,24956328690132	-35,21121578197935
2	sao jose da mata	PB	-7,181092335882639	-35,984763356596574
3	sao domingos do cariri	PB	-7,633933993213537	-36,434754536393676
4	sao jose dos cordeiros	PB	-7,3908048281076155	-36,80645019319417
5	sao joao do tigre	PB	-8,082587165509773	-36,851440344888715
6	sao jose dos cordeiros	PB	-7,390796482094682	-36,80637194932293
7	sao jose dos cordeiros	PB	-7,390271108391588	-36,80720987412586
8	sao joao do cariri	PB	-7,390455230060922	-36,533877820399475
9	sao jose dos cordeiros	PB	-7,391347125656721	-36,80565421989927
10	sao joao do cariri	PB	-7,389135386929374	-36,530930710268656
11	sao joao do cariri	PB	-7,38891159065695	-36,529747868605874
12	sao jose dos cordeiros	PB	-7,445126999999946	-36,85919299999995
13	sao mamede	PB	-6,927377151418117	-37,09180860972421
14	sao mamede	PB	-6,9257674957890645	-37,096418042718476
15	sao mamede	PB	-6,925735354441645	-37,096782730141896
16	sao jose do sabugi	PB	-6,775896970030418	-36,79834595097977
17	sao jose do bonfim	PB	-7,158442898593389	-37,30572903555816
18	sao jose de caiana	PB	-7,251735660852433	-38,30001773062324
19	sao jose do bonfim	PB	-7,1621684759242665	-37,30944149999999
20	sao jose de princesa	PB	-7,7404634999999935	-38,098183500000005

Source:

- SQL01
- SQL02

Phase 2: Sales & Inventory Performance

[Question 4]: Investors are asking about our "Heavy Hitters." What are our Top 20 best-selling products? Make sure to include the Product ID, Category, Volume (in), and total revenue generated so I can see if size affects sales.

Code:

```
--- Q4
SELECT TOP 20
        ordit.product_id,
        pro.product_category_name,
        pro.product_height_cm * pro.product_length_cm * pro.product_width_cm as
volume, --preguntar si es en in o cm
        COUNT(ordit.product_id) as n_sold,
        SUM(ordit.price) as totalrev
FROM OrderItems as ordit JOIN Products as pro on ordit.product_id = pro.product_id
GROUP by ordit.product_id,
        pro.product_category_name,
        pro.product_height_cm * pro.product_length_cm * pro.product_width_cm
ORDER BY n_sold DESC
```

Result:

```sql
--- Q4 ??
SELECT TOP 20
        ordit.product_id,
        pro.product_category_name,
        pro.product_height_cm * pro.product_length_cm * pro.product_width_cm as volume, --preguntar si es
        COUNT(ordit.product_id) as n_sold,
        SUM(ordit.price) as totalrev
FROM OrderItems as ordit JOIN Products as pro on ordit.product_id = pro.product_id
GROUP by ordit.product_id,
        pro.product_category_name,
        pro.product_height_cm * pro.product_length_cm * pro.product_width_cm
ORDER BY n_sold DESC
```

Results Messages

	product_id	product_category_name	volume	n_sold	totalrev
1	aca2eb7d00ea1a7b8ebd4e68314663af	moveis_decoracao	15000	527	37608,90000000029
2	99a4788cb24856965c36a24e339b6058	cama_mesa_banho	20000	488	43025,560000000376
3	422879e10f46682990de24d770e7f83d	ferramentas_jardim	19800	484	26577,220000000165
4	389d119b48cf3043d311335e499d9c6b	ferramentas_jardim	32560	392	21440,590000000044
5	368c6c730842d78016ad823897a372db	ferramentas_jardim	19800	388	21056,80000000003
6	53759a2ecddad2bb87a079a1f1519f73	ferramentas_jardim	19800	373	20387,20000000002
7	d1c427060a0f73f6b889a5c7c61f2ac4	informatica_acessorios	8000	343	47214,509999999966
8	53b36df67ebb7c41585e8d54d6772e08	relogios_presentes	2288	323	37683,42000000013
9	154e7e31ebfa092203795c972e5804a6	beleza_saude	4500	281	6325,18999999996
10	3dd2a17168ec895c781a9191c1e95ad7	informatica_acessorios	4840	274	41082,60000000021
11	2b4609f8948be18874494203496bc318	beleza_saude	3960	260	22717,22000000008
12	7c1bd920dbdf22470b68bde975dd3ccf	beleza_saude	3960	231	13866,689999999959
13	a62e25e09e05e6faf31d90c6ec1aa3d1	relogios_presentes	7632	226	24051
14	5a848e4ab52fd5445cdc07aab1c40e48	*NULL*	3600	197	24229,030000000075
15	bb50f2e236e5eea0100680137654686c	beleza_saude	2992	195	63885
16	e0d64dcfaa3b6db5c54ca298ae101d05	relogios_presentes	2856	194	31786,820000000058
17	42a2c92a0979a949ca4ea89ec5c7b934	utilidades_domesticas	32994	183	10870,099999999964
18	e53e557d5a159f5aa2c5e995dfdf244b	informatica_acessorios	2210	183	15439,24999999997
19	b532349fe46b38fbc7bb3914c1bdae07	moveis_decoracao	10000	169	6162,109999999991
20	35afc973633aaeb6b877ff57b2793310	casa_conforto	21200	165	14484,889999999961

Source:

- SQL02
- SQL03
- Announcement.

[Question 5]: Our warehouse manager says some categories are overstuffed while others are empty. How many products do we have in each category? Show me the category name and the count.

Code:

```sql
--Q5
SELECT
```

```
        pro.product_category_name,
        COUNT(pro.product_category_name) as n_cat
FROM Products as pro
GROUP by pro.product_category_name
ORDER by n_cat DESC --to see what is overcrowded and undercrowded on both extremes.
```

Result:

```
38  --Q5
39  SELECT
40          pro.product_category_name,
41          COUNT(pro.product_category_name) as n_cat
42  FROM Products as pro
43  GROUP by pro.product_category_name
44  ORDER by n_cat DESC --to see what is overcrowded and undercrowded on both extremes.
45
```

Results | Messages

	product_category_name	n_cat
1	cama_mesa_banho	3029
2	esporte_lazer	2867
3	moveis_decoracao	2657
4	beleza_saude	2444
5	utilidades_domesticas	2335
6	automotivo	1900
7	informatica_acessorios	1639
8	brinquedos	1411
9	relogios_presentes	1329
10	telefonia	1134
11	bebes	919
12	perfumaria	868
13	fashion_bolsas_e_acessorios	849
14	papelaria	849
15	cool_stuff	789
16	ferramentas_jardim	753
17	pet_shop	719
18	eletronicos	517
19	construcao_ferramentas_construcao	400
20	eletrodomesticos	370

Source:

- SQL05

[Question 6]: Customers are complaining that shipping is too expensive for certain items. I want to see the average delivery price by product category. Calculate this based on the time from approval to the customer reaching their door (HINT: Use DATEDIFF).

Code:

```
--Q6 price and dates diff -> quality percieved (ime it takes) vs price from
customers
SELECT
        AVG(ordit.freight_value) as avg_freight,
        pro.product_category_name,
        AVG(DATEDIFF(day, ord.order_approved_at,
ord.order_delivered_customer_date)) as avg_datediff
FROM OrderItems as ordit JOIN Products as pro on ordit.product_id = pro.product_id
        JOIN Orders as ord on ordit.order_id = ord.order_id
GROUP by pro.product_category_name
```

Result:

```sql
--Q6 price and dates diff??? -> quality percieved (ime it takes) vs price from customers
SELECT
        AVG(ordit.freight_value) as avg_freight,
        pro.product_category_name,
        AVG(DATEDIFF(day, ord.order_approved_at, ord.order_delivered_customer_date)) as avg_datediff
FROM OrderItems as ordit JOIN Products as pro on ordit.product_id = pro.product_id
        JOIN Orders as ord on ordit.order_id = ord.order_id
GROUP by pro.product_category_name
```

Results Messages

	avg_freight	product_category_name	avg_datediff
1	21,10653594771242	artigos_de_natal	15
2	19,220363164721135	eletrodomesticos	10
3	23,5938880706922	eletroportateis	10
4	18,821781014437217	informatica_acessorios	12
5	27,409544117647055	instrumentos_musicais	12
6	16,78116007344352	relogios_presentes	12
7	17,567462121212124	telefonia_fixa	12
8	19,354880382775114	artes	10
9	19,255116279069767	artigos_de_festas	8
10	22,30117781402937	bebes	11
11	22,901059602649013	casa_construcao	12
12	20,1365625	dvds_blu_ray	12
13	18,774312977099235	fashion_calcados	15
14	14,627404580152673	fashion_underwear_e_moda_praia	13
15	40,551123595505636	moveis_escritorio	20
16	22,72468013468014	climatizacao	11
17	17,438891820580466	consoles_games	12
18	11,938749999999999	fashion_roupa_infanto_juvenil	8
19	12,830833333333333	livros_importados	7
20	16,62842676311031	livros_interesse_geral	10

Source:

- SQL04
- Announcement.

Phase 3: Strategic Growth & Cleanup

[Question 7]: We're doing a "Portfolio Clean-up." Show me the 3 best-selling and the 3 worst-selling product categories specifically over the last 2 years recorded in the system. We might need to drop the losers.

Code:

```
SELECT * FROM (
SELECT top 3
        pro.product_category_name,
        COUNT(ordit.product_id) as n_sold,
        SUM(ordit.price) as totalrev
FROM OrderItems as ordit
        JOIN Products as pro on ordit.product_id = pro.product_id
        JOIN Orders as ord on ordit.order_id = ord.order_id
WHERE YEAR(ord.order_purchase_timestamp) >= (select
YEAR(MAX(ord.order_purchase_timestamp)) FROM Orders as ord) - 2
GROUP by pro.product_category_name
ORDER BY n_sold DESC  ) top3

--union all
https://stackoverflow.com/questions/5551064/combining-order-by-and-union-in-sql-ser
ver -> https://stackoverflow.com/a/5551093
UNION ALL

SELECT * FROM (
SELECT top 3
        pro.product_category_name,
        COUNT(ordit.product_id) as n_sold,
        SUM(ordit.price) as totalrev
FROM OrderItems as ordit
        JOIN Products as pro on ordit.product_id = pro.product_id
        JOIN Orders as ord on ordit.order_id = ord.order_id
WHERE YEAR(ord.order_purchase_timestamp) >= (select
YEAR(MAX(ord.order_purchase_timestamp)) FROM Orders as ord) - 2
GROUP by pro.product_category_name
ORDER BY n_sold ASC ) worst3
```

Result:

```sql
--Q7 ??? -> ver anuncio union all -> lectures only use specific hardcoded dates use -> https://stackoverflow.com/questions/6

SELECT * FROM (
SELECT top 3
        pro.product_category_name,
        COUNT(ordit.product_id) as n_sold,
        SUM(ordit.price) as totalrev
FROM OrderItems as ordit
        JOIN Products as pro on ordit.product_id = pro.product_id
        JOIN Orders as ord on ordit.order_id = ord.order_id
WHERE YEAR(ord.order_purchase_timestamp) >= (select YEAR(MAX(ord.order_purchase_timestamp)) FROM Orders as ord) - 2
GROUP by pro.product_category_name
ORDER BY n_sold DESC  ) top3

--union all https://stackoverflow.com/questions/5551064/combining-order-by-and-union-in-sql-server -> https://stackoverflow.
UNION ALL

SELECT * FROM (
SELECT top 3
        pro.product_category_name,
        COUNT(ordit.product_id) as n_sold,
        SUM(ordit.price) as totalrev
FROM OrderItems as ordit
        JOIN Products as pro on ordit.product_id = pro.product_id
        JOIN Orders as ord on ordit.order_id = ord.order_id
WHERE YEAR(ord.order_purchase_timestamp) >= (select YEAR(MAX(ord.order_purchase_timestamp)) FROM Orders as ord) - 2
GROUP by pro.product_category_name
ORDER BY n_sold ASC ) worst3
```

Results Messages

	product_category_name	n_sold	totalrev
1	cama_mesa_banho	11115	1036988,6799999896
2	beleza_saude	9670	1258681,3399999922
3	esporte_lazer	8641	988048,9699999955
4	seguros_e_servicos	2	283,28999999999996
5	fashion_roupa_infanto_juvenil	8	569,85
6	pc_gamer	9	1545,95

Source:

- SQL04
- Announcement
- https://stackoverflow.com/questions/5551064/combining-order-by-and-union-in-sql-server
- https://stackoverflow.com/questions/69209231/find-orders-in-the-last-12-months-max-date-being-the-last-order-date

---

[Question 8]: Simple growth check: Show me the total revenue by year. I need to see that upward curve!

Code:

```sql
--Q8 SQL4 challenge 2

Select
        year(ord.order_purchase_timestamp) as YearOrder,
        SUM(ordit.price) as totalrev
```

```
From     OrderItems as ordit
        join Orders as ord on ord.order_id = ordit.order_id
group by year(ord.order_purchase_timestamp)
Order by year(ord.order_purchase_timestamp) ASC
```

Result:

```
85    --Q8 SQL4 challenge 2
86
87    Select
88            year(ord.order_purchase_timestamp) as YearOrder,
89            SUM(ordit.price) as totalrev
90    From     OrderItems as ordit
91            join Orders as ord on ord.order_id = ordit.order_id
92    group by year(ord.order_purchase_timestamp)
93    Order by year(ord.order_purchase_timestamp) ASC
94
```

Results Messages

	YearOrder	totalrev
1	2016	49785,91999999997
2	2017	6155806,9800001895
3	2018	7386050,800000227

Source:

- SQL04

[Question 9]: Marketing is building a new CRM tool. They need a unique internal identifier. Create an ID that combines the zip code and city name for every customer. Show me the Customer ID, State, and this new combined ID.

Code:

```
--Q9 SQL4 combine texts -> Create an ID that combines the zip code and city name
for every customer. Show me the Customer ID, State, and this new combined ID.
-- join integer and string:
https://stackoverflow.com/questions/52937888/how-to-convert-data-int-to-string-whil
e-joining-tables-in-sql

SELECT
        cus.customer_id,
        cus.customer_state,
        CAST(cus.customer_zip_code_prefix as varchar) + ', ' + cus.customer_city as
unique_id
FROM Customers as cus
```

Result:

```
96   --Q9 SQL4 combine texts -> Create an ID that combines the zip code and city name for every customer. Show m
97   -- join integer and string: https://stackoverflow.com/questions/52937888/how-to-convert-data-int-to-string-
98
99   SELECT
100         cus.customer_id,
101         cus.customer_state,
102         CAST(cus.customer_zip_code_prefix as varchar) + ', ' + cus.customer_city as unique_id
103  FROM Customers as cus
104
```

Results Messages

	customer_id	customer_state	unique_id
1	06b8999e2fba1a1fbc88172c00ba8bc7	SP	14409, franca
2	18955e83d337fd6b2def6b18a428ac77	SP	9790, sao bernardo do campo
3	4e7b3e00288586ebd08712fdd0374a03	SP	1151, sao paulo
4	b2b6027bc5c5109e529d4dc6358b12c3	SP	8775, mogi das cruzes
5	4f2d8ab171c80ec8364f7c12e35b23ad	SP	13056, campinas
6	879864dab9bc3047522c92c82e1212b8	SC	89254, jaragua do sul
7	fd826e7cf63160e536e0908c76c3f441	SP	4534, sao paulo
8	5e274e7a0c3809e14aba7ad5aae0d407	MG	35182, timoteo
9	5adf08e34b2e993982a47070956c5c65	PR	81560, curitiba
10	4b7139f34592b3a31687243a302fa75b	MG	30575, belo horizonte
11	9fb35e4ed6f0a14a4977cd9aea4042bb	MG	39400, montes claros
12	5aa9e4fdd4dfd20959cad2d772509598	RJ	20231, rio de janeiro
13	b2d1536598b73a9abd18e0d75d92f0a3	SP	18682, lencois paulista
14	eabebad39a88bb6f5b52376faec28612	SP	5704, sao paulo
15	1f1c7bf1c9b041b292af6c1c4470b753	RS	95110, caxias do sul
16	206f3129c0e4d7d0b9550426023f0a08	SP	13412, piracicaba
17	a7c125a0a07b75146167b7f04a7f8e98	RJ	22750, rio de janeiro
18	c5c61596a3b6bd0cee5766992c48a9a1	SP	7124, guarulhos
19	9b8ce803689b3562defaad4613ef426f	SP	5416, sao paulo
20	49d0ea0986edde72da777f15456a0ee0	PA	68485, pacaja

Source:

- SQL04
- https://stackoverflow.com/questions/52937888/how-to-convert-data-int-to-string-while-joining-tables-in-sql

---

Phase 4: Customer Satisfaction & VIPs

[Question 10]: We've had a PR nightmare in the coastal regions. Show me the worst 5 reviews in Bombinhas and the worst 10% of reviews in Ipanema. I need the customer and product IDs so Support can send them an "I'm Sorry" coupon.

Code:

```
--Q10  worst 5 reviews in Bombinhasand the worst 10% of reviews in Ipanema. I need
the customer and product IDs -> need review_score, customer_id, product_id
-- customers -> city
-- orders -> customer id and order id
-- orderitems -> order id and product id
-- orderreviews -> review score by order id
-- use union all

SELECT * FROM (
SELECT top 5
        ordrev.review_score,
        cus.customer_id,
        ordit.product_id,
        cus.customer_city
FROM Orders as ord
        join Customers as cus on ord.customer_id = cus.customer_id
        join  OrderItems as ordit on ordit.order_id = ord.order_id
        join OrderReviews as ordrev on ord.order_id = ordrev.order_id
WHERE cus.customer_city = 'bombinhas'
order by ordrev.review_score asc) as worst5_bombinhas

UNION ALL

SELECT * FROM (
SELECT top 10 PERCENT
        ordrev.review_score,
        cus.customer_id,
        ordit.product_id,
        cus.customer_city
FROM Orders as ord
        join Customers as cus on ord.customer_id = cus.customer_id
        join  OrderItems as ordit on ordit.order_id = ord.order_id
        join OrderReviews as ordrev on ord.order_id = ordrev.order_id
WHERE cus.customer_city = 'ipanema'
order by ordrev.review_score asc) as worst10p_ipanema
```

Result:

```sql
--Q10  worst 5 reviews in Bombinhasand the worst 10% of reviews in Ipanema. I need the customer and product I
-- customers -> city
-- orders -> customer id and order id
-- orderitems -> order id and product id
-- orderreviews -> review score by order id
-- use union all

SELECT * FROM (
SELECT top 5
        ordrev.review_score,
        cus.customer_id,
        ordit.product_id,
        cus.customer_city
FROM Orders as ord
        join Customers as cus on ord.customer_id = cus.customer_id
        join  OrderItems as ordit on ordit.order_id = ord.order_id
        join OrderReviews as ordrev on ord.order_id = ordrev.order_id
WHERE cus.customer_city = 'bombinhas'
order by ordrev.review_score asc) as worst5_bombinhas

UNION ALL

SELECT * FROM (
SELECT top 10 PERCENT
        ordrev.review_score,
        cus.customer_id,
        ordit.product_id,
        cus.customer_city
FROM Orders as ord
        join Customers as cus on ord.customer_id = cus.customer_id
        join  OrderItems as ordit on ordit.order_id = ord.order_id
        join OrderReviews as ordrev on ord.order_id = ordrev.order_id
WHERE cus.customer_city = 'ipanema'
order by ordrev.review_score asc) as worst10p_ipanema
```

Results Messages

	review_score	customer_id	product_id	customer_city
1	1	6ba6e5bf30ec197d1fefdebdc67ffa21	de35b12d754d29dc6d48ae30cfe1c23f	bombinhas
2	1	bfcc9aef1f24975cce422915f48b0b56	1750753e77dc38c4afe6fc90f02c39bc	bombinhas
3	3	170078dbc688ee0902d9ec412728d3c7	e7d9d7787c96a30e0f418993c1555633	bombinhas
4	4	29f6bd1b03efb0a649ed019fc4157ca3	acdd408fe971bbe29a81f8febb575ffc	bombinhas
5	4	b7e08e844dc1417cf4f338c401fef8c6	77049be2845b00694201083dce7e1f2b	bombinhas
6	1	5f87c05477db2d22c63b718299c9c03e	86fb7ef6ee1ffa3832f09b1a1552e7e6	ipanema

Source:

- SQL03

[Question 11]: The UX team wants to move away from raw numbers. Create a 5-step Likert scale for our review scores—convert them to labels from "extremely bad" to "extremely good." Show the Review Score, the new Score Name, the Customer, and the Product.

Code:

```sql
--Q11 SQL5
--Show the Review Score, the new Score Name, the Customer, and the Product
```

```sql
SELECT
        ordrev.review_score,
        case when ordrev.review_score in (1) then 'extremely bad'
                when ordrev.review_score in (2) then 'bad'
                when ordrev.review_score in (3) then 'neutral'
                when ordrev.review_score in (4) then 'good'
                else  'extremely good'
                end as rev_cat,
        cus.customer_id,
        ordit.product_id
FROM Orders as ord
        join Customers as cus on ord.customer_id = cus.customer_id
        join  OrderItems as ordit on ordit.order_id = ord.order_id
        join OrderReviews as ordrev on ord.order_id = ordrev.order_id
```

Result:

```sql
--Q11 SQL5
--Show the Review Score, the new Score Name, the Customer, and the Product
SELECT
        ordrev.review_score,
        case when ordrev.review_score in (1) then 'extremely bad'
                when ordrev.review_score in (2) then 'bad'
                when ordrev.review_score in (3) then 'neutral'
                when ordrev.review_score in (4) then 'good'
                else  'extremely good'
                end as rev_cat,
        cus.customer_id,
        ordit.product_id
FROM Orders as ord
        join Customers as cus on ord.customer_id = cus.customer_id
        join  OrderItems as ordit on ordit.order_id = ord.order_id
        join OrderReviews as ordrev on ord.order_id = ordrev.order_id
```

Results Messages

	review_score	rev_cat	customer_id	product_id
1	5	extremely good	ed9339b395f5aad0302afac0f30a0c6f	ac44def10e24998cdee20a208d691127
2	5	extremely good	55d413947e32a818250f7dd43cf39704	43423cdffde7fda63d0414ed38c11a73
3	4	good	3e0806f0201dbf64cf2c8f6a6422ef38	a703f5ade6e4fae527357132230ea778
4	5	extremely good	d40a1ebd5fb73cb6fc9950aabd029157	a18a23a783fb7576a8f0f6fbc2ae2b58
5	3	neutral	7ca3b9ef13b93f0aeea91fc136aa9d18	4ce9ab528124f89e091b17d11aa2e97c
6	2	bad	bfa37f3d076b32972189c3dc3dde75b7	6b0510f16f7767cece59352e08e5432b
7	2	bad	bfa37f3d076b32972189c3dc3dde75b7	561e48cdfacd14e7cfe67aa4ebd0caea
8	2	bad	bfa37f3d076b32972189c3dc3dde75b7	a8624848cc85066425b656121d3da966
9	4	good	a1e5248dbcbe64e29f9c09f723033a36	3ba2b0de14984ce8f072c58be8d701dd
10	5	extremely good	d6184e6a56d0baf7b891bee8f43c5066	908dd61b840a480fa931ef04389c6266
11	5	extremely good	b0c1002c1c2785950d4753721642b0be	ef7f13c7694be202c48017b71801815c
12	4	good	9298659a888542ee1dccd284384d8ff7	b206760d97dbc76527bb07029a6179cb
13	5	extremely good	49a6ae8a95c6a78d90945b983ab1ecfc	59536eeb13a52ef81966f81b91fb481e
14	5	extremely good	49a6ae8a95c6a78d90945b983ab1ecfc	59536eeb13a52ef81966f81b91fb481e
15	5	extremely good	49a6ae8a95c6a78d90945b983ab1ecfc	59536eeb13a52ef81966f81b91fb481e
16	5	extremely good	49a6ae8a95c6a78d90945b983ab1ecfc	7001d71d1ad858e07e5a341649412e76
17	5	extremely good	44b6e4637c442ca2e33a5fc2b9b7a339	4fcb3d9a5f4871e8362dfedbdb02b064
18	1	extremely bad	aca9f3d4889f7141d94840387c9e3d5a	202bd859659a841de892b00c341300ff
19	4	good	a6d8c2fa92836346256c28d885d9eea0	7f9e40aa498f8f6edf4cc51215ecce9e
20	4	good	a6d8c2fa92836346256c28d885d9eea0	8d28eb3a7309368ca0de19e016266251

Source:

- SQL05

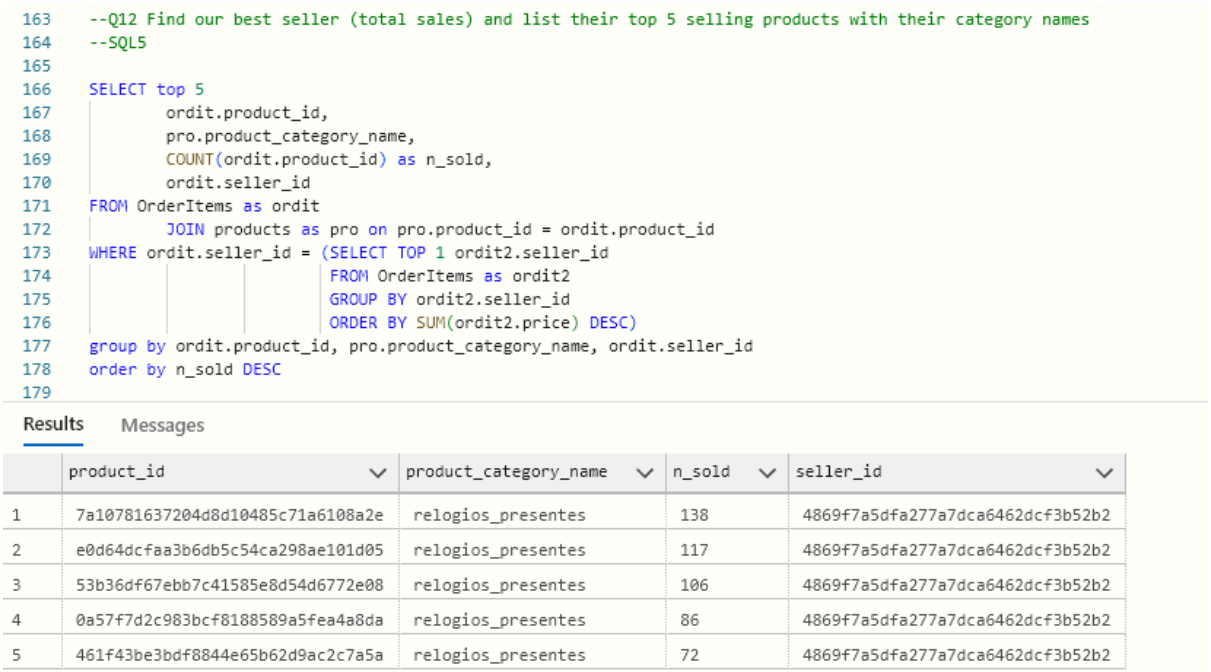
[Question 12]: We want to feature our "Star Seller" in the annual report. Find our best seller (total sales) and list their top 5 selling products with their category names.

Code:

```sql
--Q12 Find our best seller (total sales) and list their top 5 selling products with
their category names
--SQL5

SELECT top 5
        ordit.product_id,
        pro.product_category_name,
        COUNT(ordit.product_id) as n_sold,
        ordit.seller_id
FROM OrderItems as ordit
        JOIN products as pro on pro.product_id = ordit.product_id
WHERE ordit.seller_id = (SELECT TOP 1 ordit2.seller_id
                         FROM OrderItems as ordit2
                         GROUP BY ordit2.seller_id
                         ORDER BY SUM(ordit2.price) DESC)
group by ordit.product_id, pro.product_category_name, ordit.seller_id
order by n_sold DESC
```

Result:



Results | Messages

	product_id	product_category_name	n_sold	seller_id
1	7a10781637204d8d10485c71a6108a2e	relogios_presentes	138	4869f7a5dfa277a7dca6462dcf3b52b2
2	e0d64dcfaa3b6db5c54ca298ae101d05	relogios_presentes	117	4869f7a5dfa277a7dca6462dcf3b52b2
3	53b36df67ebb7c41585e8d54d6772e08	relogios_presentes	106	4869f7a5dfa277a7dca6462dcf3b52b2
4	0a57f7d2c983bcf8188589a5fea4a8da	relogios_presentes	86	4869f7a5dfa277a7dca6462dcf3b52b2
5	461f43be3bdf8844e65b62d9ac2c7a5a	relogios_presentes	72	4869f7a5dfa277a7dca6462dcf3b52b2

Source:

- SQL05